

Are LLMs Good at Mathematical Reasoning?

A study of reasoning techniques for Large Language Models on grade-school math word problems.

Group 12 · Aayush Ranjan (2021003) · Arnav Agarwal (2021235) · Harshvardhan Singh (2021052) · Parthiv Dholaria (2021078) · Pulkit Nargotra (2021273) · Utsav Garg (2021108)

1. What this project is, in one paragraph

Modern LLMs write fluent English but are unreliable at multi-step arithmetic reasoning. This project asks a simple question — *how much can you improve an LLM's math reasoning without retraining it?* — and answers it empirically. We evaluate four open-weight models (LLaMA 3.1 70B, LLaMA 3.1 8B, Gemma 2 9B, Mixtral 8x7B) on two math word-problem benchmarks (GSM8K, SVAMP) across a ladder of increasingly sophisticated techniques: zero-shot prompting → chain-of-thought → ReAct with a calculator tool → self-correction → parameter-efficient fine-tuning → retrieval-augmented generation → multi-agent collaboration → **our proposed RAG + Multi-Agent framework**, which combines the last two and is the main contribution.

The headline result: the RAG + Multi-Agent framework reaches **100% accuracy with LLaMA 3.1 70B** on the 100-sample GSM8K and SVAMP test subsets, versus ~87% for plain zero-shot prompting. The headline caveat: when we perturb the test questions with irrelevant details, that same framework **falls apart**, which suggests a lot of the apparent "reasoning" is pattern-matching against training data.

2. Why this problem matters

Financial analysis, education technology, and robotics all need models that can carry a chain of arithmetic through several steps without drifting. Fluency is not enough — a model that is 90% right on multi-step problems is wrong on nearly every 10-step task. Understanding *where* LLMs break, and which scaffolding actually fixes it versus merely papering over it, is a prerequisite for deploying them anywhere the arithmetic matters.

3. Datasets

Dataset	What it is	Size used	Role
GSM8K (Grade School Math 8K)	8,000 high-quality grade-school word problems requiring multi-step arithmetic, algebra, and logic	7,473 train / 1,319 val; 100-sample test subset for the expensive methods	Primary benchmark
SVAMP (Single Variable Arithmetic Math Problems)	Word problems built to test whether a model actually parses the question rather than pattern-matching surface structure	700 train / 300 val; 100-sample test subset	Secondary benchmark
AQUA-RAT	Multiple-choice math problems with worked explanations	1,000 items indexed	Knowledge base only — never evaluated on. Retrieved from during RAG.
GSM-Hard	GSM8K with the numbers swapped for large, uncommon values	Sampled	Stress test for numerical generalisation

Why the test sets are only 100 samples: every method here is an API call per problem, and multi-agent methods are 4+ calls per problem. Groq/Together credit limits capped the evaluation. This is the single biggest limitation of the results below — treat the accuracy numbers as indicative, not as tight estimates. A 100-sample accuracy has roughly a ± 5 –10 point confidence interval.

4. How accuracy is measured

1. **Ground truth** answers are extracted from the dataset.
2. The model is instructed to end its response with `Answer: <<<Numerical Answer>>>`, and a **regex** pulls that number out.
3. Responses that don't match the format, or that produce no number, are marked **invalid and excluded from the denominator**.
4. A prediction counts as correct only on an **exact numerical match**.

$$\text{Accuracy} = (\text{Number of Correct Predictions} / \text{Total Valid Predictions}) \times 100$$

Read this carefully: excluding invalid responses from the denominator means a model that formats badly is *not* penalised for it. Two models with the same reported accuracy may have very different usable-output rates. Blank cells in the baseline table below are due to inconsistent output formats, exhausted API credits, or compute constraints — not zero performance.

5. The methods, explained

5.1 Baselines

Zero-shot prompting. Ask for the answer, get the answer. No reasoning steps, no examples. Establishes the floor.

Chain-of-Thought (CoT). Add "think step by step" — the model decomposes the problem into intermediate steps before answering. The classic Wei et al. technique.

ReAct (Reasoning + Acting). Interleave reasoning with tool calls. We attach LangChain's `llm-math` calculator so the model can offload arithmetic. Sounds like it should win; it doesn't (see §6) — the tool helps with *calculation* but GSM8K's difficulty is in *decomposition*.

Self-correction. A three-round prompting loop:

- *Round 1* — model answers the question; a second pass extracts the number.
- *Round 2* — "Review your previous answer and find problems with it."
- *Round 3* — "Based on the problems you found, improve your answer."

Multi-Agent (Teacher-Student). A Student LLM solves; a Teacher LLM critiques and returns corrective feedback; the loop repeats until convergence on a templated final answer.

5.2 Fine-tuning (mid-review phase)

Three parameter-efficient approaches, all on the theory that task-specific adaptation would beat prompting. **All three failed**, which is itself a finding.

Method	How it works	Config
QLoRA	Quantise base weights to 4/8-bit, freeze them, train small low-rank matrices	4 epochs, LR 5e-4, r=32, targets q_proj, k_proj, v_proj, dense
IA³	Freeze weights entirely; learn scaling vectors applied to activations	4 epochs, LR 5e-4, targets q_proj, k_proj, v_proj, dense, fc1, fc2
Prompt tuning	Freeze the model; learn 20 soft prompt-token embeddings prepended to the input	4 epochs, LR 5e-4, 20 virtual tokens
Prefix tuning	Learn a 10-token prefix in the key/value cache	3 epochs, LR 1e-3, weight decay 1e-4

LoRA runs used **Fireworks.ai** (hosted LoRA); prefix/prompt tuning ran on **Kaggle** GPUs.

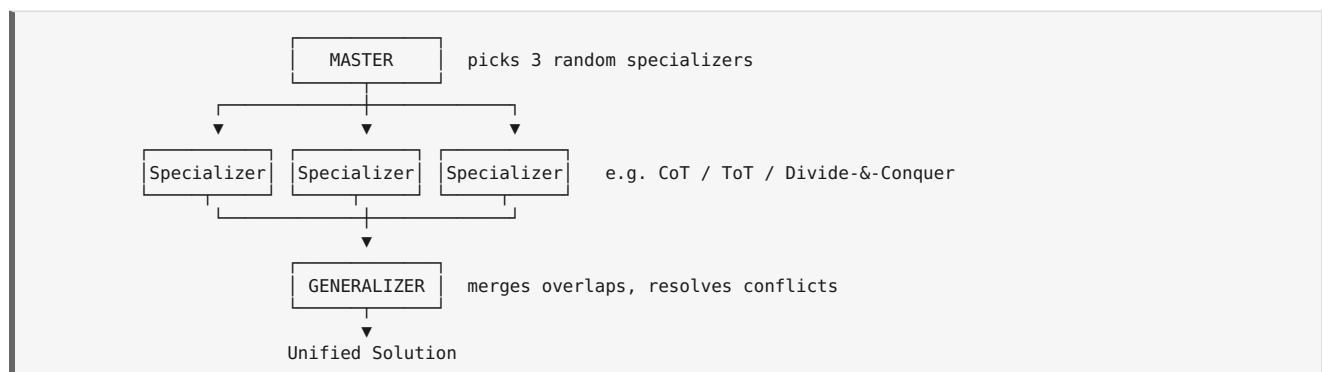
5.3 RAG (Retrieval-Augmented Generation)

- Embed problems with **SentenceTransformer** `all-MiniLM-L6-v2`.
- Index 1,000 **AQUA-RAT** question/question/option/answer/explanation records in a **Pinecone** vector DB.
- At inference: embed the problem → retrieve **top-3** nearest question-answer pairs → prepend them to the prompt → generate.

An important honest observation from the paper: *this behaves like few-shot prompting*. It retrieves similar solved examples and pastes them in. It is not doing knowledge lookup in any deep sense.

5.4 Multi-Agent Framework (Specializers) — the core contribution

Inspired by Meta-Reasoning Prompting (Gao et al., 2024), which selects a reasoning method per task. Our version runs several reasoning styles in parallel and merges them.



Seven specializers, each a distinct prompt encoding one reasoning style:

Specializer	Strategy
Chain of Thought	Sequential steps, verify each before moving on
Tree of Thought	Explore multiple solution paths, pick the best
Step-Back	Generalise to a simpler question first, then specialise back
Analogical Reasoning	Find a well-known similar problem, adapt its solution
Divide-and-Conquer	Split into sub-problems, solve independently, recombine
Solo Performance Prompting	Simulate a panel of personas (Calculator / Logical Thinker / Verifier)
Hypothesis Testing	Enumerate candidate solutions, test each against the constraints

Roles: the *Master* selects three specializers and collects their answers; the *Generalizer* synthesises them into one coherent solution. Responses are generated through the **Groq** API.

5.5 RAG + Multi-Agent — the combined framework

Same as above, with one change: a **RAG-based specializer** is added to the pool and is made a **mandatory participant**, joined by two randomly chosen reasoning specializers. So every problem gets retrieved exemplars *plus* two independent reasoning styles, all merged by the Generalizer. This is the configuration that produces the best numbers.

5.6 Neuro-symbolic solvers (attempted, not adopted)

Following Logic-LM: have the LLM translate the word problem into algebraic equations as strings, then solve them with **SymPy**. We abandoned this. Four failure modes:

- **Extraction** — regexes reliably miss some of the generated equations.
- **Problem diversity** — the number and form of equations varies wildly, so no fixed parser works.
- **Over-generation of variables** — the LLM invents unnecessary variables, leaving the system unsolvable.
- **Non-determinism** — the model often just solves the equations itself instead of emitting them, defeating the point.

6. Results

6.1 Baselines (full test sets)

Method	SVAMP — Mixtral	SVAMP — Gemma	SVAMP — Llama	GSM8K — Mixtral	GSM8K — Gemma	GSM8K — Llama
Zero Shot	64.13	85.91	90.55	72.41	88.96	87.32
CoT (Zero shot)	75.68	89.53	89.68	72.72	89.93	87.54
ReAct	84.61	73.33	79.87	67.57	57.45	75.21
Multi-Agent (Teacher-Student)	66.00	90.00	92.00	–	–	–
Before Self-Correction	77.68	90.42	86.22	77.01	85.71	83.41
After Self-Correction	72.90	85.63	84.21	74.71	83.52	80.46

Blank cells: inconsistent output formats, insufficient credits, or resource constraints.

What to notice:

- **CoT helps, modestly.** Every model improves over zero-shot, but Llama barely moves (90.55 → 89.68 on SVAMP) — it was likely already reasoning step-by-step unprompted.
- **ReAct is erratic.** It's the best method for Mixtral on SVAMP (84.61) and among the worst for Gemma (73.33). On GSM8K it hurts everything. Tool access helps with arithmetic; GSM8K's difficulty is multi-step decomposition, which a calculator doesn't touch.
- **Self-correction makes things worse, consistently.** All six model/dataset pairs drop. This replicates Huang et al. (2023) — *intrinsic* self-correction without external feedback causes models to talk themselves out of correct answers.

6.2 Advanced methods (100-sample subsets)

RAG alone

Dataset	Gemma	Mistral	LLaMA 8B
GSM8K	88%	53%	81%
SVAMP	71%	68%	81%

Multi-Agent alone

Dataset	LLaMA 70B	LLaMA 8B	Mistral	Gemma
SVAMP	95%	93%	80%	89%
GSM8K	95%	87%	65%	92%

RAG + Multi-Agent (the proposed framework)

Dataset	Gemma	Mistral	LLaMA 70B	LLaMA 8B
SVAMP	92%	88%	100%	84%
GSM8K	93%	78.35%	100%	88%

What to notice:

- **Multi-agent beats RAG alone for every model.** Diversity of reasoning strategies matters more than retrieved exemplars.
- **Model size dominates.** LLaMA 70B leads in every configuration.
- **Mistral is the weak link** — 53% on GSM8K under RAG. It handles retrieved context poorly.
- **Gemma is the most consistent** across all setups, making it the pragmatic mid-size choice.
- **100% on a 100-sample subset should not be read as "solved."** It means zero errors on 100 problems drawn from a benchmark the model may well have seen in pre-training — which is exactly what §7.1 investigates.

6.3 Fine-tuning results (mid-review)

LoRA, via Fireworks.ai:

Model	SVAMP	GSM8K
LLaMA 3.1 8B Instruct	62.67%	41.00%
Mixtral 8x7B Instruct	57.00%	48.42%
Gemma 7B	not supported	not supported

Prompt tuning and prefix tuning both **increased hallucination**. Training loss decreased steadily while outputs became unrelated to the prompt.

Every fine-tuning result is worse than plain zero-shot prompting. Compare 41% (LoRA, LLaMA, GSM8K) against 87.32% (zero-shot, same model, same dataset). The diagnosis is **catastrophic forgetting** — the models drift away from their pre-trained capabilities. Contributing factors:

- **Limited scope** — LoRA updates a small parameter subset, insufficient for deep reasoning tasks.
- **Task mismatch** — low-rank adaptation isn't the right lever for multi-step problem-solving.
- **Objective drift** — soft prompts and prefixes pulled the model off its causal-LM objective.
- **Too few epochs** — GPU and memory limits capped training.

This is the clearest lesson in the project: **for reasoning tasks, inference-time scaffolding beat parameter-efficient fine-tuning by a wide margin.**

7. Where it breaks — the honest part

7.1 Membership inference: is the model reasoning or remembering?

100% accuracy on GSM8K raised the obvious suspicion of training-data contamination. We perturbed test problems in two ways, using RAG + Multi-Agent on LLaMA 3.1 8B Instruct.

Perturbation A — swap names and numbers, keep the logic.

Original: Toulouse has twice as many sheep as Charleston. Charleston has 4 times as many sheep as Seattle. How many sheep do Toulouse, Charleston, and Seattle have together if Seattle has 20 sheep?
Perturbed: Tory has thrice as many sheep as Danny. Danny has 5 times as many sheep as Jimmy. How many sheep do Tory, Charlie, and Jimmy have together if Jimmy has 18 sheep?

Result: robust. The model kept solving these, indicating it tracks logical structure rather than memorising specific surface forms.

Perturbation B — inject irrelevant details.

Original: Jean has 30 lollipops. Jean eats 2. With the remaining lollipops, Jean wants to package 2 per bag. How many bags can Jean fill?
*Perturbed: Damien has 20 lollipops. **Damien is also a huge fan of cookies and would prefer eating cookies any day.** Damien eats 4 of the lollipops **because there are no cookies.** With the remaining, he wants 4 per bag. How many bags can he fill?*
Correct answer: 4. Framework's answer: "3 bags with 4 lollipops each and 1 bag with 4 lollipops."

The model computed $16 \div 4 = 4$ correctly, then invented a reason to reject its own result.

*Second example: a ping-pong scoring problem with "**Mike eventually won by a huge margin of 8 points and went on to win the championship**" appended. The framework correctly summed $8 + 10 + 2 = 20$, then **added the 8-point victory margin** to reach 28.*

Result: significant failure. The framework cannot filter irrelevant information. It compulsively incorporates every number it sees. Failures cluster where the distractor superficially resembles a relevant quantity.

This aligns with Apple's 2024 finding that LLMs may not be performing genuine logical reasoning so much as probabilistic matching against training data. **The 100% figure and this failure mode have to be read together.**

7.2 GSM-Hard: large numbers break the arithmetic

GSM-Hard replaces GSM8K's numbers with large, uncommon values.

Setting	LLaMA 3.1 70B
Zero-shot	48%
RAG + Multi-Agent	53%

The framework helps (+5 points) but performance is poor in absolute terms. Inspecting failures: **the reasoning steps were usually correct, but the model couldn't execute the arithmetic**. Answers land near the right value but wrong. The conclusion is that these models need an external calculator for genuine numerical work — a symbolic or tool-augmented layer, not more prompting.

7.3 Model scale comparison (70B vs 8B)

Aspect	LLaMA 70B	LLaMA 8B
GSM8K accuracy	87.4%	76.2%
SVAMP accuracy	82.5%	71.8%
Error rate, complex multi-step	9%	19%
Successful revision under RAG feedback	85%	65%
Retrieval-based question accuracy	90.1%	79.3%

Larger models process longer contexts, hold more reasoning steps, and integrate retrieved knowledge better.

8. Conclusions

1. **RAG + Multi-Agent is the best-performing method tested** — 100% with LLaMA 70B on both benchmark subsets, and improvements for every model.
2. **Multi-agent contributes more than retrieval**. Diverse reasoning strategies outperform retrieved exemplars in every comparison.
3. **Self-correction without external feedback hurts**. Consistently, across all models and datasets.
4. **Parameter-efficient fine-tuning failed badly** on these reasoning tasks — catastrophic forgetting, worse than zero-shot.
5. **Scale still dominates**. 70B beats 8B everywhere.
6. **Robustness is the unsolved problem**. Irrelevant details and large numbers both break the framework, suggesting a mix of pattern-matching and genuine reasoning rather than reasoning alone.

9. Future work

- **Symbolic solver augmentation** — pair the framework with SymPy or similar to handle the arithmetic the model can't (directly addresses §7.2).
- **Broader benchmarks** — MATH, AQUA-RAT, GAME-OF-24, and adversarial sets with deliberately misleading details.
- **Evaluator feedback loop** — an evaluator module that critiques intermediate outputs, giving the *extrinsic* feedback that §6.1 shows self-correction lacks.
- **Domain-specialised agents** — fine-tune individual agents for algebra, geometry, commonsense; allocate tasks dynamically by expertise.
- **Meta-reasoning** — predict which reasoning chain will succeed for a given problem instead of picking three at random. This would also give the framework a mechanism for ignoring irrelevant details.

10. Repository guide

```
.
├── Group_12_Baseline_Results/      # Phase 1: baseline prompting techniques
├── Group12_MidReview_Results/     # Phase 2: fine-tuning experiments
└── Final Submission/              # Phase 3: RAG, multi-agent, and the combined framework
```

Group_12_Baseline_Results/ — baseline prompting

File	What it does
Zero Shot Prompting.ipynb	Zero-shot baseline, LLaMA 3.1 8B Instant via Groq
ZeroShotWithGroq.ipynb	Zero-shot across Gemma 2 9B and LLaMA 3.1 8B
Chain of Thought Prompting.ipynb	CoT prompting baseline
project-llm-gsm8k.ipynb	GSM8K driver — LangChain harness across Groq/Mistral/Anthropic backends, all three models
llm-project-svamp.ipynb	SVAMP equivalent of the above
llama_self_correction_final.ipynb	Three-round self-correction, LLaMA
gemma_self_correction_final.ipynb	Three-round self-correction, Gemma
mistral_self_correction_final.ipynb	Three-round self-correction, Mistral
MultiAgentWithGroq.ipynb	Early teacher-student multi-agent, Gemma + Mistral
MultiAgentWithMultiModel.ipynb	Multi-agent with heterogeneous models as student and teacher

Group12_MidReview_Results/ — fine-tuning

File	What it does
LORA Fine tuning/ LORA_FineTuning_Dataset.ipynb	Builds the LoRA training sets from GSM8K/SVAMP
LORA Fine tuning/LORA_Finetuning.ipynb	Fireworks.ai LoRA job submission and polling
LORA Fine tuning/ LORA_Finetuning_LLAMA_GSM.ipynb	LoRA — LLaMA 3.1 8B on GSM8K
LORA Fine tuning/ LORA_Finetuning_Mistral.ipynb	LoRA — Mistral on SVAMP
LORA Fine tuning/ LORA_FineTuning_MISTRAL_GSM8K.ipynb	LoRA — Mistral on GSM8K
LORA Fine tuning/settings_*.yaml	Fireworks configs — base model, input/output templates. settings_llama.yaml, settings_llama_gsm8k.yaml, settings_mistral.yaml, settings_mistral_gsm8k.yaml
LORA Fine tuning/*_train.jsonl, *_val.jsonl	Training data. GSM8K: 7,473 train / 1,319 val. SVAMP: 700 train / 300 val
LORA Fine tuning/correct_progress*.txt	Running correctness logs from evaluation
peft-llm (1).ipynb	PEFT prompt tuning — Gemma 7B IT
peft-llm-mistral.ipynb	PEFT prompt tuning — Mistral 7B
prompt-tuning-llama (1).ipynb	Prompt tuning — LLaMA 3.1 8B, 20 virtual tokens
prefix-gemma.ipynb	Prefix tuning — Gemma 7B IT
prefix-mistral.ipynb	Prefix tuning — Mistral 7B
prefix-tuning-llama (1).ipynb	Prefix tuning — LLaMA 3.1 8B

Final Submission/ — the main experiments

FineTuning/ — all three use `meta-llama/Meta-Llama-3-8B-Instruct` with the PEFT + TRL stack, and each logs `before_finetuning` / `after_finetuning` outputs to JSONL for comparison:

File	Method
<code>llm-project-finetuning-qlora.ipynb</code>	QLoRA — 4 epochs, LR 5e-4, r=32
<code>llm-project-finetuning-ia3.ipynb</code>	IA ³ — activation scaling vectors
<code>llm-project-finetuning-prompt-tuning.ipynb</code>	Prompt tuning — 20 virtual tokens

MultiAgent/ — the seven-specializer framework without retrieval:

File	Model
<code>MultiAgent_70B_gsm8k_svamp.ipynb</code>	LLaMA 3.1 70B Versatile, both datasets
<code>MultiAgent_Gemma.ipynb</code>	Gemma 2 9B IT
<code>multiagent-gemma.ipynb</code>	Gemma 2 9B, Pinecone-enabled variant
<code>multiagent-8b.ipynb</code>	LLaMA 3.1 8B Instruct
<code>multiagent-mistral (2).ipynb</code>	Mixtral 8x7B Instruct

MultiAgent_with_RAG/ — the proposed framework. The `stacked-llms*` notebooks are the earlier stacked-LLM formulation; the `multi*agent*rag*` notebooks are the final version with the mandatory RAG specializer:

File	Model / dataset
<code>multiagent_rag_70b_gsm8k_svamp.ipynb</code>	LLaMA 3.1 70B — the 100% result
<code>multi_agent_rag_llama_8b_gsm8k.ipynb</code>	LLaMA 3.1 8B on GSM8K
<code>rag-multiagent-gemma9b.ipynb</code>	Gemma 2 9B
<code>stacked-llms_metallama_70b_gsm8k.ipynb</code>	LLaMA 70B, stacked formulation, GSM8K
<code>stacked-llms_gemma_9b_gsm8k.ipynb</code>	Gemma 9B, stacked, GSM8K
<code>stacked-llms-mixtral_gsm8k.ipynb</code>	Mixtral, stacked, GSM8K
<code>stacked-llms-svamp-mixtral.ipynb</code>	Mixtral, stacked, SVAMP

RAG/ — recorded per-problem outputs from the Gemma RAG runs, each entry containing `index`, `question`, `original_answer`, and `model_output`:

File	Contents
<code>gsm8k_rag_gemma_multiline.json</code>	GSM8K RAG outputs
<code>svamp_rag_gemma_multiline.json</code>	SVAMP RAG outputs

Note: these two files have stray integer tokens between objects and are not valid JSON as written. Strip the tokens between `}` and `{` before parsing.

11. Running the code

Requirements

```
pip install groq transformers datasets torch peft \
    sentence-transformers pinecone-client \
    langchain langchain-groq langchain-mistralai \
    pandas numpy scikit-learn tqdm
```

External services

Service	Used for	Where
Groq	Primary inference (LLaMA, Gemma, Mixtral)	Multi-agent and baseline notebooks
Pinecone	Vector store for the AQUA-RAT knowledge base	All RAG notebooks
HuggingFace	Model and tokenizer downloads	Fine-tuning notebooks
Fireworks.ai	Hosted LoRA fine-tuning	LoRA Fine tuning/
Kaggle	GPU compute for prefix/prompt tuning	Mid-review notebooks

Model identifiers used: `llama-3.1-70b-versatile`, `llama-3.1-8b-instant`, `meta-llama/Llama-3.1-8B-Instruct`, `gemma2-9b-it`, `google/gemma-2-9b-it`, `mixtral-8x7b-32768`, `mistralai/Mixtral-8x7B-Instruct-v0.1`, `all-MiniLM-L6-v2` (embeddings).

API keys

The notebooks originally contained hardcoded API keys. These have been **replaced with environment-variable lookups** before publication. Set the following before running anything:

```
export GROQ_API_KEY="your-groq-key"
export PINECONE_API_KEY="your-pinecone-key"
export HF_TOKEN="your-huggingface-token"
```

The notebooks read them as:

```
import os
groq_api_key = os.environ.get("GROQ_API_KEY")
pinecone_api_key = os.environ.get("PINECONE_API_KEY")
hf_token = os.environ.get("HF_TOKEN")
```

Some notebooks were written for Kaggle and use `kaggle_secrets.UserSecretsClient()` instead; either works — set whichever your environment supports. If you are running on Kaggle, add the keys under *Add-ons* → *Secrets*.

Note that a few cell **outputs** in older notebooks may still be worth reviewing before re-running — exception tracebacks in Jupyter can echo function arguments, which is how keys leaked into saved output in the first place. Clearing outputs before committing (`jupyter nbconvert --clear-output --inplace notebook.ipynb`) avoids this.

Suggested reading order

1. `Group_12_Baseline_Results/Zero Shot Prompting.ipynb` — the floor
2. `Group_12_Baseline_Results/Chain of Thought Prompting.ipynb` — first improvement
3. `Final_Submission/MultiAgent/MultiAgent_70B_gsm8k_svamp.ipynb` — the specialist architecture
4. `Final_Submission/MultiAgent_with_RAG/multiagent_rag_70b_gsm8k_svamp.ipynb` — the full framework and headline result

12. Reference documents

The zip archive this repository was assembled from also contains (not committed here, code files only):

- `LLM_Project_Final.pdf` — the 39-page final report, source for all results above
- `G12 MID REVIEW Report.pdf` — fine-tuning results and catastrophic-forgetting analysis
- `Group12_BaselineResults.pdf` — baseline write-up
- `Group_12_survey_paper.pdf` — literature survey (26 references: LogicBench, ProofWriter, PrOntoQA, Tree of Thoughts, Logic-LM, DiLA, LoGiPT, ROSCOE, RECEVAL, and others)
- `LLM Final Presentation.pptx`, `Survey Paper Presentation.pptx` — slide decks

13. Key references

- Wei et al., *Chain of Thought Prompting Elicits Reasoning in LLMs*, arXiv:2201.11903
- Yao et al., *Tree of Thoughts*, arXiv:2305.10601
- Huang et al., *Large Language Models Cannot Self-Correct Reasoning Yet*, arXiv:2212.07919
- Gao et al., *Meta-Reasoning for Large Language Models*, arXiv:2406.11698 — direct inspiration for the specializer framework
- Pan et al., *Logic-LM*, arXiv:2305.12295 — the neuro-symbolic approach attempted in §5.6
- Rasal, *LLM Harmony: Multi-Agent Communication for Problem Solving*, arXiv:2401.01312
- Wang et al., *Solo-Performance Prompting*, arXiv:2307.05300